Digital VLSI Design

4-bit Array Multiplier

From RTL to GDSII with Open-Source Tools

Team Members

Name	**Roll Number**
Veekshita Sai	2024102026
Chandini Gayathri	2024102020

Abstract

This report describes a 4-bit unsigned array multiplier and its full journey from Verilog code to a finished layout (GDSII) on the SkyWater SKY130 130 nm process, using only open-source tools. The design was checked with all 256 input combinations. It was then synthesized with Yosys into 73 SKY130 standard cells (16 flip-flops, 805.773 μm^2 of cell area), and the gate-level netlist gave the same results as the RTL. Static timing analysis with OpenSTA at the typical corner gave a minimum clock period of 2.596 ns, which is a maximum frequency of about 385 MHz. Place and route with LibreLane took eight runs. The first seven runs failed setup timing in the slow signoff corner. At a clock period of 7.0 ns the design passed with +0.335 ns of slack, zero DRC errors and zero LVS errors. The post-route maximum frequency was about 150 MHz, and the core area was 17,759.5 μm^2. The report explains each step, the numbers, and why the results change as the design gets closer to real silicon.

Contents

1 Introduction **3**

2 Objectives **3**

3 Design Overview: how a 4-bit array multiplier works **3**
3.1 Partial products . . . 3
3.2 The array structure . . . 3
3.3 Size and delay . . . 4
3.4 Registers, latency and throughput . . . 4

4 Design specification **5**

5 Technology and tools **5**

6 Methodology **6**

7 Part A: RTL design and verification **6**
7.1 Overview . . . 6
7.2 Design approach . . . 6
7.3 The testbench . . . 9
7.4 Results . . . 9
7.5 Observations . . . 10

8 Part B: Logic synthesis **10**
8.1 Overview . . . 10
8.2 The idea behind it . . . 11
8.3 The synthesis script . . . 11
8.4 Results . . . 12
8.5 Checking that synthesis did not change the behaviour . . . 12
8.6 Observations . . . 13

9 Part C: Static timing analysis **13**
9.1 Overview . . . 13
9.2 The idea behind it . . . 13
9.3 The OpenSTA script . . . 14
9.4 Analysis and results . . . 15
9.5 Comparing with the estimate of about $2N$ full-adder delays . . . 16
9.6 Observations . . . 17

10 Part D: Physical design (RTL to GDSII) **17**
10.1 Overview . . . 17
10.2 The idea behind it . . . 17
10.3 Configuration . . . 18
10.4 Finding a working clock period . . . 19
10.5 Final results (run 8, 7.0 ns) . . . 21
10.6 Cell count: after synthesis and after place and route . . . 21
10.7 The layout . . . 22
10.8 Post-route F_{max} compared with Part C . . . 23
10.9 Observations . . . 24

11 Answers to the questions **24**

12 Conclusion **26**

1 Introduction

Multiplication is one of the most common operations inside a digital chip. It is used in DSP blocks, graphics, machine-learning accelerators and the arithmetic unit of almost every processor. A multiplier is also a good example for learning digital design. It has long carry chains, a clear critical path, and a direct trade-off between area and speed.

We designed a 4×4 unsigned array multiplier that gives an 8-bit product. An array multiplier is the hardware version of the multiplication we do by hand. It forms all the partial products at once and then adds them row by row using a regular grid of full adders. The structure is simple and regular, so it is easy to understand and easy to lay out, even though it is not the fastest multiplier.

The RTL-to-GDSII flow converts a hardware design written in a hardware description language (HDL) into the physical layout that is needed for manufacturing. It has several stages: logic synthesis, floorplanning, placement, clock-tree synthesis, routing, and layout checking. The flow needs electronic design automation (EDA) tools and a process design kit (PDK). With the open-source SKY130 PDK and tools such as Yosys, OpenSTA, OpenROAD, Magic, KLayout, Netgen and LibreLane, the whole flow can now be run on a normal laptop. This lets us see how the cell count, area and timing change from one stage to the next.

2 Objectives

1. Design a 4-bit array multiplier in Verilog using a full-adder module instantiated 12 times, with registered inputs and a registered output, and test it with a self-checking testbench.
2. Synthesize the RTL into SKY130 standard cells with Yosys, report the cell count and area, and show with gate-level simulation that synthesis did not change the behaviour.
3. Run static timing analysis with OpenSTA to find the minimum clock period, the maximum frequency and the critical path.
4. Take the design through floorplanning, placement, clock tree synthesis, routing and signoff with LibreLane, and end with a DRC-clean and LVS-clean layout that meets timing.
5. Compare the numbers from each stage and explain why they are different.

3 Design Overview: how a 4-bit array multiplier works

3.1 Partial products

Multiplying two binary numbers A and B uses the same shift-and-add method as decimal long multiplication. For every bit b_i of B we make one partial product. It is A ANDed with that bit and shifted left by i places. Adding the four partial products gives the 8-bit result. In binary this is easy, because each bit of a partial product is just one AND gate.

3.2 The array structure

In hardware, the four partial products are added one row at a time:

- Row 0 has only AND gates. Its lowest bit is already a final product bit (`p[0]`), and its other three bits go to row 1.

- Rows 1, 2 and 3 each have four AND gates and a chain of four full adders, which works as a 4-bit ripple-carry adder. Row i adds its partial product to the bits that come from the row above.
- The lowest sum bit of each of these rows is a finished product bit (`p[1]`, `p[2]`, `p[3]`). The other three sum bits and the last carry go to the next row.
- The last row also gives the upper four bits of the product, `p[7:4]`.

Here $pp_i[j] = a[j]$ AND $b[i]$, and the carry-in of the first adder of every row is 0. The exact connections are in Table 1.

Table 1: Connections of the array (j = 0 to 3).

Row	Inputs to full adder j	Outputs
0	no adders	`p[0] = pp0[0]`; passes down `r0 = {0, pp0[3], pp0[2], pp0[1]}`
1	`pp1[j]`, `r0[j]`, carry `c1[j]` (`c1[0] = 0`)	sum `s1[j]`, carry `c1[j+1]`; `p[1] = s1[0]`; passes down `r1 = {c1[4], s1[3], s1[2], s1[1]}`
2	`pp2[j]`, `r1[j]`, carry `c2[j]` (`c2[0] = 0`)	sum `s2[j]`, carry `c2[j+1]`; `p[2] = s2[0]`; passes down `r2 = {c2[4], s2[3], s2[2], s2[1]}`
3	`pp3[j]`, `r2[j]`, carry `c3[j]` (`c3[0] = 0`)	sum `s3[j]`, carry `c3[j+1]`; `p[3] = s3[0]`; `p[7:4] = {c3[4], s3[3], s3[2], s3[1]}`

3.3 Size and delay

An N-bit array multiplier uses N^2 AND gates and $N(N-1)$ full adders. For $N = 4$ this is 16 AND gates and 12 full adders. The longest path goes diagonally through the array, so its delay is about $2N$ full-adder delays. So the delay grows about linearly with N, while the area grows with N^2.

3.4 Registers, latency and throughput

The multiplier has registers on both its inputs and its output, so all timing is measured from one flip-flop to the next. Inputs applied before clock edge 1 are captured at edge 1, and the product is captured into the output register at edge 2. So the latency is 2 clock cycles. A new input pair can be accepted on every clock edge, so the throughput is one result per cycle, which is equal to the clock frequency.

4 Design specification

Table 2: Ports of the module `mult_array`.

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low; clears all registers to 0
`a`	input	4	Multiplicand
`b`	input	4	Multiplier
`p`	output	8	Product $a \times b$, registered

The design has these fixed properties:

- 16 flip-flops: 4 for `a`, 4 for `b` and 8 for `p`.
- 16 AND gates and 12 `full_adder` instances (three rows of four). The `*` operator is not used in the RTL.
- Latency of 2 clock cycles, and throughput of one result per cycle.
- Module, port and file names are exactly as the assignment says.

5 Technology and tools

Process and cell library. The design uses the SkyWater SKY130 130 nm process with the high-density standard cell library `sky130_fd_sc_hd`. The timing analysis in Part C used the typical corner, `tt_025C_1v80` (25°C, 1.8 V). The LibreLane signoff in Part D checks several corners. These include a slow corner at 100°C and 1.6 V (`ss_100C_1v60`) and a fast corner at −40°C and 1.95 V (`ff_n40C_1v95`). This turned out to matter a lot for the final clock period.

Table 3: Tools used at each stage.

Stage	Tool (version)	Used for
RTL and gate-level simulation	Icarus Verilog 14.0	Compile and run the testbench
Waveform viewing	GTKWave 4.0.0	Look at `mult.vcd` and show the latency
Logic synthesis	Yosys 0.70	RTL to SKY130 gate-level netlist
Static timing analysis	OpenSTA 3.1.0	Slack, T_{min}, critical path
RTL-to-GDSII flow	LibreLane (with OpenROAD, Magic, Netgen)	Floorplan, placement, CTS, routing, signoff
Layout viewing	KLayout 0.30.12	Look at the GDSII and make layout images
Environment	IIC-OSIC-TOOLS Docker image	All tools and the PDK in one container

Working environment. We worked on an Ubuntu Linux laptop. We ran the IIC-OSIC-TOOLS Docker image (`hpretl/iic-osic-tools:latest`) in its browser (VNC) mode.

This gives a full Linux desktop in a browser tab, with every tool and the PDK already installed. The project folder is shared between the laptop (`/eda/designs`) and the container (`/foss/designs`), so our files are safe when the container stops. One small point: the image uses a different PDK by default (`ihp-sg13g2`), so in every new terminal we had to run `sak-pdk sky130A` first.

6 Methodology

The multiplier was built and checked in four stages. Each stage uses the results of the one before it. The flow goes from RTL design and testing, to logic synthesis, to static timing analysis, and finally to physical design (Table 4).

Table 4: Stages of the RTL-to-GDSII flow.

Stage	Tool	Output
A: RTL design and verification	Icarus Verilog, GTKWave	Simulation results and waveforms
B: Logic synthesis	Yosys	Gate-level netlist and cell area report
C: Static timing analysis	OpenSTA	Timing report, slack values and critical path
D: Physical design	LibreLane	GDSII layout and result numbers

The same self-checking testbench was used at the RTL level and at the gate level. If both give the same result, synthesis kept the behaviour of the design.

7 Part A: RTL design and verification

7.1 Overview

In this stage we wrote the multiplier in Verilog and checked that it gives the right answer for every possible input. Each input has only 4 bits, so there are just $16 \times 16 = 256$ combinations. We could test all of them instead of only a few. A design that passes this test has no untested cases at the functional level, so we can trust the later stages (synthesis, timing, layout) more.

7.2 Design approach

We built the design from two modules, as the assignment asks.

`full_adder` is a one-bit adder with inputs `a`, `b` and `cin`, and outputs `sum` and `cout`. The sum is 1 when an odd number of the inputs are 1, so it is the XOR of all three. The carry is 1 when at least two inputs are 1, so it is a majority function. Both are written with `assign` statements:

```
module full_adder (
    input  a, b, cin,
    output sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

`mult_array` is the multiplier. It follows the row structure of Section 3 and has three parts:

1. **Input registers.** `a` and `b` are stored in 4-bit registers `a_r` and `b_r` on every rising clock edge. They use an asynchronous, active-low reset (`rst_n`) that clears them to 0.

2. **The array.** The logic works on `a_r` and `b_r`. One AND mask per row forms the partial products, which gives the 16 AND gates. Row 0 has no adders. Rows 1, 2 and 3 each use a `generate` loop with four `full_adder` instances and a carry chain, so there are 12 full adders in total. Each row passes its upper three sum bits and its last carry to the next row (`r0`, `r1`, `r2`).

3. **Output register.** The 8 product bits are collected in a signal `p_next` and stored in the output register `p` on the next clock edge, with the same asynchronous reset.

One row of the array looks like this in the code (this is row 1; rows 2 and 3 have the same form):

```
wire [4:0] c1;
wire [3:0] s1;
assign c1[0] = 1'b0;
genvar j;
generate
    for (j = 0; j < 4; j = j + 1) begin : row1
        full_adder fa (.a(pp1[j]), .b(r0[j]), .cin(c1[j]),
                       .sum(s1[j]), .cout(c1[j+1]));
    end
endgenerate
wire [3:0] r1 = {c1[4], s1[3], s1[2], s1[1]};
```

The product bits are taken directly from Table 1:

```
wire [7:0] p_next = {c3[4], s3[3], s3[2], s3[1], s3[0], s2[0], s1[0], pp0[0]};
```

- We used a `generate` loop for each row instead of writing 12 separate instances. The code is shorter, there is less chance of a typing mistake in a wire name, and a mistake in one row is easy to see because all rows look the same.

- The `*` operator is not used anywhere in the design, so the multiplier is really built from AND gates and full adders. The only place where it appears is the testbench, where it gives the expected answer.

- Registers on both the inputs and the output mean that all timing is measured from flip-flop to flip-flop. This keeps the timing analysis in Part C clean, and it also sets the latency to 2 cycles.

Figure 1 shows the complete structure of `mult_array`: the input registers, all 16 AND gates, all 12 full adders and the output register. Each row of full adders is drawn one step to the right of the row above it. In this way every sum bit goes straight down into the adder below it, and every carry goes to the right inside its row. AND gate (i, j) makes $pp_i[j] = a_r[j] \cdot b_r[i]$, and FA_{ij} is the full adder in row i, bit j. The product bit $p[k]$ comes out of column k.

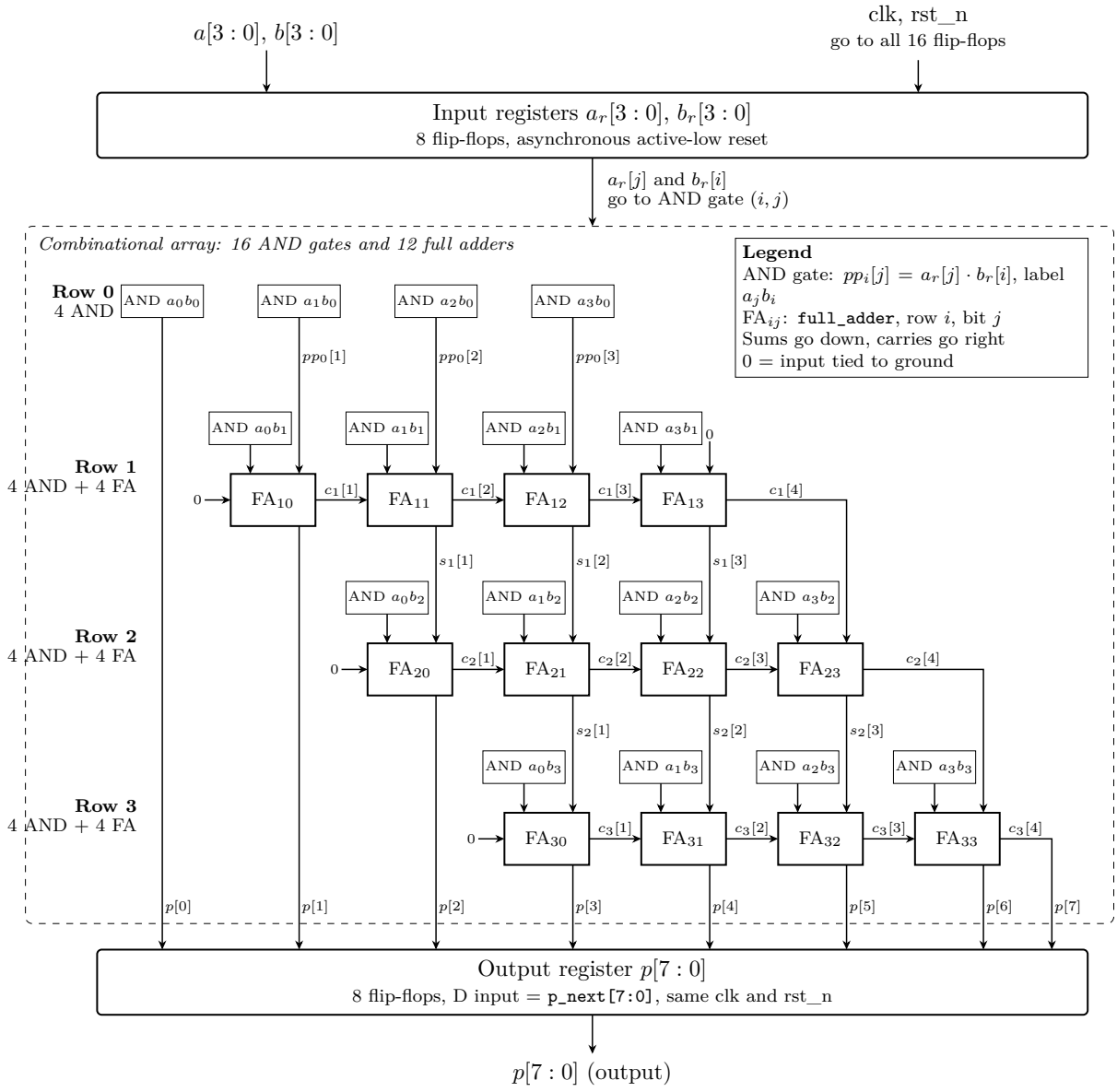


Figure 1: Block diagram of `mult_array`: input registers, 16 AND gates, 12 full adders (rows 1 to 3) and the output register. Sums go down, carries go to the right, and product bit $p[k]$ comes out of column k.

7.3 The testbench

The testbench `tb_mult.v` is self-checking. It decides by itself if the design passed, so nobody has to look at waveforms for 256 cases. It works like this:

- **Clock and reset.** A clock with a 10 ns period is generated. `rst_n` is held low for the first two cycles and then released on a falling edge.
- **Applying inputs.** For every pair (a, b) with a and b from 0 to 15, the inputs are applied on a falling clock edge. This keeps them away from the rising edge, so there is no race between the testbench and the design.
- **Waiting for the latency.** The testbench waits for two rising edges (the 2-cycle latency) and then looks at the output.
- **Checking.** It compares `p` with the expected value `a * b` using `!==`. This also catches unknown (X) values, which a plain `!=` would miss. For every mismatch it prints a line with a, b, the value seen and the value expected.
- **Summary.** At the end it prints exactly one summary line, either `PASS: 256 vectors checked, 0 errors` or `FAIL: 256 vectors checked, N errors`. It also writes the waveform file `mult.vcd`.

Each vector takes 20 ns (two clock cycles), so the whole run finishes at about 5140 ns. The design can accept a new input pair on every cycle. We used two cycles per vector only to keep the check easy to follow.

7.4 Results

We compiled and ran the simulation with:

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
```

The result was:

```
PASS: 256 vectors checked, 0 errors
```

Checking that the testbench can fail. A testbench that always prints PASS would be useless, so the assignment asks us to break the design on purpose. We changed the sum equation in `full_adder.v` to `a ^ b`, which removes the carry-in term, and ran the testbench again. It printed:

```
FAIL: 256 vectors checked, 95 errors
```

For example, with $a = 3$ and $b = 3$ the broken design gave 5 instead of 9, and with $a = 3$ and $b = 7$ it gave 9 instead of 21. After this we put the original line back and ran the testbench again. It printed PASS, so the file was restored correctly.

Hand-worked example. The assignment says to take A as the last digit of a team member's roll number plus 5. We used Veekshita's roll number, 2024102026. Its last digit is 6, so $A = 6 + 5 = 11$ (binary 1011). We used $B = 7$ (binary 0111). The multiplication by hand is:

```
        1 0 1 1    A  = 11
      x 0 1 1 1    B  = 7   (b0 = 1, b1 = 1, b2 = 1, b3 = 0)
    ---------------
```

```
          1 0 1 1     pp0 = A AND b0  (b0 = 1)
        1 0 1 1       pp1 = A AND b1  (b1 = 1), shifted 1
      1 0 1 1         pp2 = A AND b2  (b2 = 1), shifted 2
    0 0 0 0           pp3 = A AND b3  (b3 = 0), shifted 3
      ---------------
  0 1 0 0 1 1 0 1     P = 77
```

As a check, $11 + 22 + 44 + 0 = 77$, and $01001101_2 = 64 + 8 + 4 + 1 = 77$.

Finding it in the waveform. In the testbench, vector number $k = 16a + b$ is applied at time $20 + 20k$ ns. For $a = 11$ and $b = 7$ we get $k = 183$, so the inputs change at **3680 ns**. The first rising edge after that, at **3685 ns**, stores them in the input registers. The second rising edge, at **3695 ns**, stores the product in the output register, and `p` becomes 77. Figure 2 shows this. `b` changes to 7 at 3680 ns. `p` changes from 66 (the result of the previous vector, 11×6) to 77 at 3695 ns, and then to 88 (11×8) at 3715 ns.

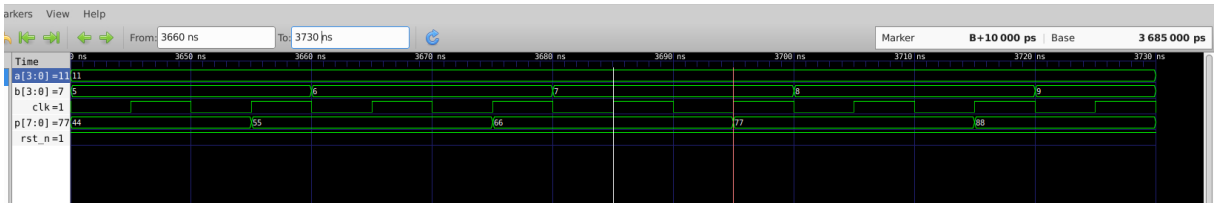


Figure 2: Waveform of the 11×7 example (`clk`, `rst_n`, `a`, `b` and `p`, shown in decimal). The two markers are on the rising clock edges at 3685 ns and 3695 ns. The product 77 appears two clock edges after the inputs, which is the 2-cycle latency.

7.5 Observations

- **Full coverage.** All 256 combinations passed, including 0×anything and $15 \times 15 = 225$, the largest product, which uses all 8 output bits.
- **Latency is clear.** In the waveform, `p` is always two rising clock edges behind `b`, as the design intends.
- **Reset works.** While `rst_n` is low, the registers are cleared and `p` stays at 0. After the reset is released, the first result follows the same two-cycle pattern.
- **The testbench is sensitive.** Breaking one equation in one module gave 95 failures out of 256, so a real mistake would not go unnoticed. This matters later, because the same testbench is our proof that synthesis kept the behaviour.

8 Part B: Logic synthesis

8.1 Overview

Part A gave us a correct design, but it is only a description of what the circuit does. It does not say which gates are used. Synthesis decides that. In this part we turned the RTL into a netlist of real SKY130 standard cells, looked at the number of cells and the area, and checked that the netlist still behaves like the RTL.

8.2 The idea behind it

A synthesis tool works in two stages.

1. **Generic synthesis.** The tool reads the RTL and changes it into a network of generic logic operations (AND, XOR, flip-flop and so on) that do not belong to any technology. It also cleans up the logic by removing constants and repeated parts.

2. **Technology mapping.** The tool then rewrites that network using only the cells that exist in the target library. Each cell has a fixed function, area and delay, which are listed in a Liberty (`.lib`) file. The tool chooses cells so that the circuit stays correct while keeping area and delay low.

Flip-flops and logic gates are mapped separately. In Yosys, `dfflibmap` maps the flip-flops and `abc` maps everything else.

We also used **flattening**. Our RTL has a hierarchy (12 `full_adder` instances inside `mult_array`). Flattening removes the borders between modules, so the tool sees the whole multiplier as one block of logic. This lets it simplify logic across the adders, which it could not do if each adder stayed a closed box.

8.3 The synthesis script

The assignment lists seven steps. Each step is one Yosys command inside the Tcl script `synth/synth.tcl`:

```
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib

yosys read_verilog rtl/full_adder.v rtl/mult_array.v
yosys synth -top mult_array -flatten
yosys dfflibmap -liberty $lib
yosys abc -liberty $lib
yosys opt_clean -purge
yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib
yosys write_verilog -noattr synth/mult_array_netlist.v
```

Table 5: What each step of `synth.tcl` does.

Step	Command	What it does
1	`read_verilog`	Reads both RTL files
2	`synth -top mult_array -flatten`	Generic synthesis with `mult_array` as the top module; flattens the full adders into one module
3	`dfflibmap -liberty`	Maps each flip-flop to a SKY130 flip-flop cell (here, one with asynchronous active-low reset)
4	`abc -liberty`	Maps all the other logic to SKY130 gates
5	`opt_clean -purge`	Removes unused wires and cells, and drops internal names
6	`tee -o ... stat -liberty`	Writes the cell and area report to a file
7	`write_verilog -noattr`	Writes the gate-level netlist without Yosys attributes

The Liberty file is the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, the same one we use later for timing. We ran the script with `yosys -c synth/synth.tcl`.

8.4 Results

Table 6: Part B results.

Metric	Value
Total number of cells	73
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	805.773 μm^2
Flip-flop area	400.384 μm^2 (16 $\times$ 25.024 μm^2)
Flip-flop area (% of total)	49.69%
Three most-used cell types	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 11, `xnor2_1` $\times$ 4 (tied at 4 with `a21oi_1`, `and2_0` and `nand4_1`)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

To see what the 73 cells are, we grouped them by function:

Table 7: The 73 synthesized cells grouped by function.

Cell group	Cells	Area (μm^2)	Share
Flip-flops (`dfrtp_1`)	16	400.384	49.7%
XOR / XNOR / majority (`xnor2`, `xnor3`, `xor3`, `maj3`)	8	113.860	14.1%
NAND (`nand2`, `nand3`, `nand4`)	16	71.319	8.9%
AND (`and2`, `and3`, `and4`)	6	40.038	5.0%
NOR (`nor2`, `nor2b`, `nor3`)	3	15.015	1.9%
OR (`or3`)	1	6.256	0.8%
Complex AND-OR-INVERT and OR-AND-INVERT gates (`a21oi`, `a21boi`, `o21ai`, `o31a`, `o2111a`, and others)	19	136.379	16.9%
Inverter (`clkinv_1`)	1	3.754	0.5%
Low-power-flow cells (`lpflow_inputisolp`, `lpflow_isobufsrc`)	3	18.768	2.3%
Total	73	805.773	100%

So the logic is 57 cells with an area of 405.389 μm^2, which is about half of the total area. The flip-flops are the other half.

8.5 Checking that synthesis did not change the behaviour

Synthesis should keep the behaviour, but tools can have bugs, so we checked. We used the **same testbench** from Part A without any change. We replaced the RTL with the synthesized netlist and the simulation models of the SKY130 cells:

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls
```

The flag `-DFUNCTIONAL` selects the simple functional cell models, and `-DUNIT_DELAY=#1` gives them a small delay so the simulation is not completely zero-delay. The result was:

```
PASS: 256 vectors checked, 0 errors
```

This shows that for all 256 input combinations the netlist gives the same output as the specification. It is not a formal equivalence proof, which would compare the two designs mathematically.

8.6 Observations

- **All flip-flops are kept.** The netlist has exactly 16 flip-flops, the same as the $4 + 4 + 8$ in the RTL. Synthesis did not remove or merge any register.
- **The logic cell count is not** $16 + 12$**.** The RTL has 16 AND gates and 12 full adders, but the netlist has 57 logic cells. The library has full-adder cells, but none of them is in our netlist. `abc` rebuilt the logic with its own methods, so each adder became several smaller cells and some AND gates were merged into nearby gates.
- **Some adders are simpler.** The first adder of every row has a carry-in of 0, and the top adder of row 1 has a 0 on one input (the top bit of `r0`). The tool can simplify these adders. This is another reason why the cells do not match the 12 adders one to one.
- **`nand2_1` is used a lot.** It is the second most used cell, with 11 copies. It is the smallest useful gate in the library (3.754 μm^2).
- **Low-power-flow cells.** Three cells come from the low-power-flow part of the library (`lpflow_...`). By their names, they are simple gates with an inverted input, and `abc` chose them because they fit the logic. In this design they work as normal logic gates.
- **Flip-flops are half of the area.** One `dfrtp_1` is 25.024 μm^2, about 6.7 times the area of a `nand2_1`. This is explained in Question 3.
- **Names are generic.** Because of `opt_clean -purge`, the cells have names like `_118_`. In Part C we had to look in the netlist to find which register the critical path starts at.

9 Part C: Static timing analysis

9.1 Overview

Synthesis told us how big the circuit is. This part asks how fast it can run. We used OpenSTA to find the minimum clock period (T_{min}), the maximum clock frequency (F_{max}) and the critical path of the synthesized netlist. We also compared the length of the critical path with the estimate of about $2N = 8$ full-adder delays from Section 3.

9.2 The idea behind it

Why static timing analysis. A simulation only checks the input patterns that we give it, so it can miss the slowest path. Static timing analysis (STA) does not simulate. It adds the delay of every cell along every path between flip-flops, using the numbers in the Liberty file, and compares each path with the clock period. It looks at all paths at once, so the worst path cannot hide.

Terms we need.

- **Clock-to-Q delay.** After a clock edge, the output of a flip-flop needs a little time to settle.
- **Setup time.** The data at a flip-flop input must be stable for a short time before the next clock edge.
- **Data arrival time.** The time when the signal reaches the end flip-flop. It is the clock-to-Q delay of the start flip-flop plus the delays of all gates in between.
- **Data required time.** The latest time the signal may arrive. It is the clock period minus the setup time.
- **Slack.** Required time minus arrival time. Positive slack means the path meets timing, zero means it just meets it, and negative means it fails.
- **WNS (worst negative slack).** The smallest slack over all paths. It belongs to the critical path.

Finding the limit. For a path from one register to another, the arrival time and the setup time do not depend on the clock period when the clock is ideal. So the slack changes one to one with the period, and

$$T_{min} = T - \text{WNS}, \qquad F_{max} = \frac{1000}{T_{min}} \text{ MHz} \quad (T_{min} \text{ in ns}).$$

So one run with a relaxed period (we used 10 ns) is enough to find the limit. A second run at the new period then confirms that the slack is about zero.

Corner. The assignment asks for the typical corner only (`tt_025C_1v80`: 25°C, 1.8 V). So the numbers are for a typical chip in typical conditions, with an ideal clock and no wire delay.

9.3 The OpenSTA script

`sta/sta.tcl` follows the seven steps from the assignment:

```
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
set T 2.60

read_liberty $lib
read_verilog synth/mult_array_netlist.v
link_design mult_array
create_clock -name clk -period $T [get_ports clk]
set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]
set_output_delay 1.0 -clock clk [all_outputs]
report_worst_slack -max
report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt
```

Table 8: What each step of `sta.tcl` does.

Step	Command	Meaning
1	`read_liberty`	Loads the cell timing from the typical-corner library
2	`read_verilog`, `link_design`	Loads the synthesized netlist and connects it to the library cells
3	`create_clock`	Makes a clock named `clk` on the `clk` port with period T
4	`set_input_delay 1.0`	Inputs arrive 1.0 ns after the clock edge; the clock port itself is removed from the list
5	`set_output_delay 1.0`	The outside world needs the outputs 1.0 ns before the next edge
6	`report_worst_slack -max`	Prints the worst setup slack
7	`report_checks -path_delay max`	Saves the critical path report with 3 decimals

We ran it with `sta -exit sta/sta.tcl`. The `-exit` flag makes the program quit at the end instead of waiting at a prompt. The period is stored in a variable `T`, so we can change it in one place. The file in the repository has the final value $T = 2.60$ ns.

9.4 Analysis and results

Step 1: relaxed clock, $T = 10$ ns. With $T = 10$ ns, OpenSTA printed `worst slack max 7.40`. The 3-digit report gave **7.404 ns** (`report_worst_slack` prints only 2 decimals). Then

$$T_{min} = 10 - 7.404 = 2.596 \text{ ns}, \qquad F_{max} = \frac{1000}{2.596} \approx 385.2 \text{ MHz}.$$

Step 2: check at $T = T_{min}$. We set T to 2.60 ns (T_{min} rounded to 0.01 ns, as the assignment asks) and ran again. The slack was +0.004 ns, which is almost zero. This is what we expected: $2.60 - 2.596 = 0.004$.

The critical path. The worst path is a normal register-to-register path:

- **Start point:** flip-flop `_118_`. In the netlist this is the input register bit `b_r[0]` (its D input is `b[0]`).
- **End point:** flip-flop `_112_`. This is the output register bit `p[6]` (its D input is `p_next[6]`).

The delays along this path at $T = 10$ ns are in Table 9.

Table 9: The critical path of the synthesized netlist (typical corner).

Stage	Cell	Cell delay (ns)	Arrival (ns)
start	`dfrtp_1` (clock-to-Q of `b_r[0]`)	0.395	0.395
1	`and2_0`	0.212	0.607
2	`or3_1`	0.361	0.968
3	`and4_1`	0.215	1.183
4	`o2111a_1`	0.234	1.417
5	`o21ai_0`	0.216	1.634
6	`xnor3_1`	0.274	1.907
7	`maj3_1`	0.375	2.282
8	`xor3_1`	0.178	2.460
end	`dfrtp_1` (setup of `p[6]`)	−0.135	required at 9.865

The data arrives at 2.460 ns. The end flip-flop needs it by $10 - 0.135 = 9.865$ ns, so the slack is 7.404 ns. (Also, $2.460 + 0.135 = 2.595$, which agrees with $T_{min} = 2.596$ within the rounding of the report.)

The report also has a second group of paths. It is the asynchronous reset `rst_n` going to the reset pins of the flip-flops, which is checked with a **recovery** check (the reset must be released a little before the clock edge, like a setup time). Its slack was 9.228 ns at $T = 10$ ns and 1.828 ns at $T = 2.60$ ns, so it is not critical.

Latency and throughput. The latency is $2 \times T_{min} = 5.192$ ns, because a result needs two clock cycles. The throughput is one result per cycle, which is 385.2 million results per second, the same as F_{max}.

Table 10: Part C results.

Metric	Value
Worst slack at $T = 10$ ns	7.404 ns
T_{min}	2.596 ns
F_{max}	385.2 MHz
Slack at $T = T_{min}$ (run at 2.60 ns)	+0.004 ns
Critical path start point	`_118_` = `b_r[0]` (`dfrtp_1`)
Critical path end point	`_112_` = `p[6]` (`dfrtp_1`)
Number of cells on the critical path	8 logic cells (10 with the two flip-flops)
Latency $= 2 \times T_{min}$	5.192 ns
Throughput	385.2 million results per second

Cells on the critical path, in order: `and2_0` → `or3_1` → `and4_1` → `o2111a_1` → `o21ai_0` → `xnor3_1` → `maj3_1` → `xor3_1`.

9.5 Comparing with the estimate of about $2N$ full-adder delays

Section 3 says that the longest path is about $2N = 8$ full-adder delays. Our path has **8 logic cells**, so the count is the same. But the eight cells are not eight full adders.

- Only three cells, `xnor3_1`, `maj3_1` and `xor3_1`, look like adder cells (XOR gives the sum and majority gives the carry). The other five (`and2_0`, `or3_1`, `and4_1`, `o2111a_1`, `o21ai_0`) are partial-product and carry logic. The three adder-type cells take $0.274 + 0.375 + 0.178 = 0.827$ ns, which is about 40% of the 2.065 ns of logic delay.

- The average delay of one stage is $2.065/8 \approx 0.26$ ns. A ripple-carry full adder has two gate levels by itself, so eight of them in a row would be much slower.

The reason is what we saw in Part B. The tool flattened the design and mapped all the logic as one block. It merged neighbouring adder and AND logic into complex gates, so the signal passes through fewer levels. The $2N$ estimate is a good guide for a plain array, and it correctly shows that the path goes from the top row to the bottom row. But after synthesis the real delay depends on the cells that the tool chose.

9.6 Observations

- **Large margin at 10 ns.** The worst slack was 7.404 ns, so the design uses only about a quarter of a 10 ns clock period.
- **Where the time goes.** Of the 2.596 ns minimum period, 2.065 ns (about 79.5%) is logic and 0.530 ns (about 20.4%) is register overhead (0.395 ns clock-to-Q and 0.135 ns setup).
- **The path starts at `b_r[0]`.** `b_r[0]` forms the first partial product (row 0) at the top of the array, so its signal travels through the most rows to reach the upper product bits. `p[6]` is one of the upper bits made by the last row. This fits how an array multiplier is built, but since the cells no longer match the original adders one to one, this is only our interpretation.
- **Slowest cells.** In the path, `maj3_1` (0.375 ns) and `or3_1` (0.361 ns) are the slowest, and `xor3_1` (0.178 ns) is the fastest.
- **Setup only.** The assignment asks only for the setup (maximum delay) analysis, so we did not run a hold check in this part. Hold is checked later in the LibreLane signoff.
- **The numbers are optimistic.** This analysis uses the typical corner, an ideal clock and no wire delay. Real silicon is slower. Part D shows by how much.

10 Part D: Physical design (RTL to GDSII)

10.1 Overview

In Parts B and C the design was a list of gates with ideal connections. In Part D it becomes a physical object. Every cell gets a place on the die, a clock tree is built, metal wires are routed between the cells, and the result is checked against the manufacturing rules. Our goal was a finished GDSII layout with no DRC errors, no LVS errors and a setup slack of at least zero. Then we compare its real speed with the ideal speed from Part C.

10.2 The idea behind it

LibreLane runs the whole RTL-to-GDSII flow and calls Yosys, OpenROAD, Magic, KLayout and Netgen for us. The stages are in Table 11.

Table 11: Stages of the physical design flow.

Stage	What it does	What we saw in our design
Synthesis	RTL to standard cells, as in Part B	LibreLane runs its own Yosys step, so the cell count is a little different from our Part B netlist
Floorplan	Sets the chip size and builds the power grid	Fixed 150 μm $\times$ 150 μm die; the cell area is very small compared with it
Placement	Puts every cell in a legal place in the rows	Cells sit in horizontal rows; tap cells are added at regular distances
Clock tree synthesis (CTS)	Adds buffers so the clock reaches all flip-flops at nearly the same time	A small tree of 3 `clkbuf_16` cells for 16 flip-flops
Routing	Draws the metal wires between cells	2,375 μm of wire in the final route
Extraction and signoff	Measures wire resistance and capacitance, runs timing in several corners, checks DRC and LVS	This is where all the failures in our early runs came from
GDSII	Writes the final layout file	`mult_array.gds`

Three ideas are needed to read the results.

Timing corners. A chip does not behave the same in all conditions. Transistors are slower at high temperature and low supply voltage, and faster at low temperature and high voltage. LibreLane therefore checks timing at several combinations: a typical corner (`tt_025C_1v80`), a slow corner (`ss_100C_1v60`) and a fast corner (`ff_n40C_1v95`). For setup, the slow corner usually fails first, because the gates are slowest there.

What DRC and LVS check. DRC (design rule check) compares the layout shapes with the rules of the foundry. LVS (layout versus schematic) extracts the circuit from the layout and compares it with the netlist. Antenna checks look for long metal wires that can collect charge during manufacturing and damage a transistor gate.

Cells that are not logic. Besides gates and flip-flops, a layout needs *tap cells* (connect the wells and substrate to the power rails), *fill and decap cells* (fill the empty parts of the rows) and *buffers* (clock buffers from CTS and timing-repair buffers from the optimizer).

10.3 Configuration

`pnr/config.json` has exactly the six settings that the assignment asks for:

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 7.0,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

Table 12: Meaning of the configuration keys.

Key	Meaning
`DESIGN_NAME`	The top module, `mult_array`
`VERILOG_FILES`	The RTL files, with paths relative to the config file
`CLOCK_PORT`	The port that has the clock, `clk`
`CLOCK_PERIOD`	The clock period in ns (the value we had to search for, see below)
`FP_SIZING`	`"absolute"` means we give the die size ourselves instead of letting the tool calculate it from the cell area
`DIE_AREA`	A fixed 150 μm $\times$ 150 μm die, needed because the design is very small

We run the flow from the `pnr/` folder with `librelane config.json`. Each run makes its own folder `runs/RUN_<date-time>/`, and its `final/` folder has the metrics and the GDS file. After the successful run we copied `final/metrics.json` and the GDS file into `pnr/`. The `runs/` folder is very large, so it is not in the repository.

10.4 Finding a working clock period

The starting point. The assignment says to set `CLOCK_PERIOD` to $1.2 \times T_{min}$ from Part C, rounded up to the next 0.5 ns, and to add 0.5 ns after every timing failure. With $T_{min} = 2.596$ ns, $1.2 \times 2.596 = 3.12$ ns, which rounds up to **3.5 ns**.

The first seven runs failed setup timing and the eighth run passed. All failures were setup violations in the slow signoff corner. The worst setup slack of every run is in Table 13 and Figure 3.

Table 13: The eight place-and-route runs.

Run	`CLOCK_PERIOD` (ns)	Worst setup slack (ns)	Result
1	3.5	−2.144	FAIL
2	4.0	−1.644	FAIL
3	4.5	−1.144	FAIL
4	5.0	−0.644	FAIL
5	5.5	−0.144	FAIL
6	6.0	−0.082	FAIL
7	6.5	−0.165	FAIL
8	**7.0**	**+0.335**	**PASS**

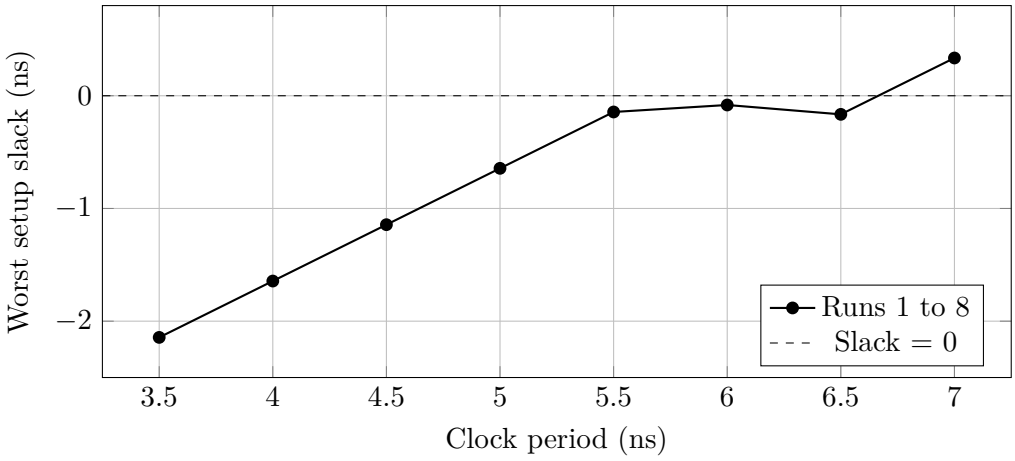


Figure 3: Worst setup slack against clock period for the eight runs. The slack improves one to one with the period up to 5.5 ns, then becomes flat and gets worse at 6.5 ns. It becomes positive only at 7.0 ns.

We ran run 1 by hand. After that, a small shell loop did the same steps for the later runs (change the period, run the flow, read the slack, write a line in the README). So every attempt is written in the README, as the assignment asks.

Why run 1 failed although Part C said 2.6 ns was enough. The report of run 1 shows the reason. At the typical corner (`tt_025C_1v80`) the worst setup slack was +0.568 ns, so the design did meet timing at 3.5 ns in the typical corner. In the slow corner (`ss_100C_1v60`) the slack was −2.144 ns. Part C looked only at the typical corner with an ideal clock and no wires. The signoff run also includes the slow corner, real wires and the clock tree, and these made the period longer.

The slack stopped improving. The first five runs behaved as expected: each 0.5 ns added to the period gave exactly 0.5 ns more slack (−2.144, −1.644, −1.144, −0.644, −0.144). After that the trend broke. From 5.5 to 6.0 ns the slack improved by only 0.062 ns, and 6.5 ns was worse than 6.0 ns (−0.165 against −0.082). We did not find the exact cause, but we have a guess. The flow synthesizes and optimizes the design again for every period, so the netlist is not the same in every run. When the clock is loose, the typical corner meets timing easily (at 6.0 ns the typical-corner slack was about +2.9 ns). Then the optimizer has little reason to make the logic faster, and some of the extra period may be lost on cells that are slower in the slow corner. This is only our guess. We saw the effect, but we did not test the cause.

What the failing path looked like. For run 7 we read the slow-corner path report. The worst path ended at the register for `p[6]` and went through adder-type cells (`a32o`, `a21boi`, `xor2`, `xnor2`, `a2bb2oi`, `maj3`, `xnor3`). One `maj3_1` took 0.89 ns in this corner, compared with 0.375 ns for the `maj3_1` in the Part C path. The report also takes away a **clock uncertainty of 0.25 ns** and uses a **setup time of 0.293 ns** (0.135 ns in Part C) from the available period.

The final run. At 7.0 ns the flow finished with a worst setup slack of +0.335 ns, so 7.0 ns is the `CLOCK_PERIOD` in the submitted `config.json`.

10.5 Final results (run 8, 7.0 ns)

Table 14: Part D results.

Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	8
Die area	22,500 μm^2 (150 $\times$ 150)
Core area	17,759.5 μm^2
Cells after synthesis (Part B)	73
Cells after place and route	1,896 in total (334 without decap and fill cells)
Kinds of cells added	245 tap, 1,313 decap, 249 fill, 3 clock buffers, about 19 timing-repair buffers
Wirelength	2,375 μm
Setup slack (worst over all corners)	+0.335 ns
Post-route $F_{max} = 1000/(7.0 - 0.335)$	150.0 MHz
F_{max} from Part C	385.2 MHz
DRC errors	0 (route, Magic and KLayout all 0)
LVS errors	0 (device, net, property, unmatched device, net and pin counts all 0)

10.6 Cell count: after synthesis and after place and route

The design grew from 73 cells to 1,896 cells. Almost all of the growth is physical filler and not logic. The numbers are from `metrics.json`, and the cell names are from the final netlists:

Table 15: Cells in the final layout.

Class	Count	Cells	What it is for
Tap cells	245	`tapvpwrvgnd_1`	Connect the n-wells and substrate to the power rails at regular distances; prevent latch-up
Decap cells	1,313	`decap_12` (904), `decap_6` (173), `decap_3` (130), `decap_8` (58), `decap_4` (48)	Fill empty row space and add capacitance between VPWR and VGND, which keeps the supply steady
Plain fill cells	249	`fill_1` (215), `fill_2` (34)	Keep the rows, power rails and layer density continuous
Clock buffers	3	`clkbuf_16`	Clock tree: a root buffer that feeds two leaf buffers (in the path report a leaf buffer drives 8 flip-flops)
Timing-repair buffers	19	mostly `buf_2`	Added by the optimizer to fix slew and setup problems on long or loaded nets
Flip-flops	16	`dfrtp_1`	Same as in Part B
Other logic	51	48 multi-input gates, 3 inverters	The combinational logic of the multiplier

The classes add up. The 245 tap cells plus the 89 “real” cells (16 flip-flops, 48 multi-input gates, 3 inverters, 19 timing-repair buffers and 3 clock buffers) make 334 standard cells. Then $334 + 1{,}562$ decap and fill cells $= 1{,}896$.

Two more points. First, the logic here (67 cells before buffering) is not the same as the 73 cells of Part B. LibreLane runs its own synthesis with its own script and target period, so its netlist is a little different. We did not use our Part B netlist in this flow. Second, the die is much bigger than the design needs. The synthesized cell area of 805.773 μm^2 is only about 4.5% of the 17,759.5 μm^2 core (a rough number, since we do not have the exact post-route cell area). This is why there are so many decap and fill cells: they fill the empty space.

10.7 The layout

Figures 4 and 5 show the final GDSII. We made them by running KLayout in batch mode with the SKY130 layer-style file from the PDK, so the layers have their usual colours.

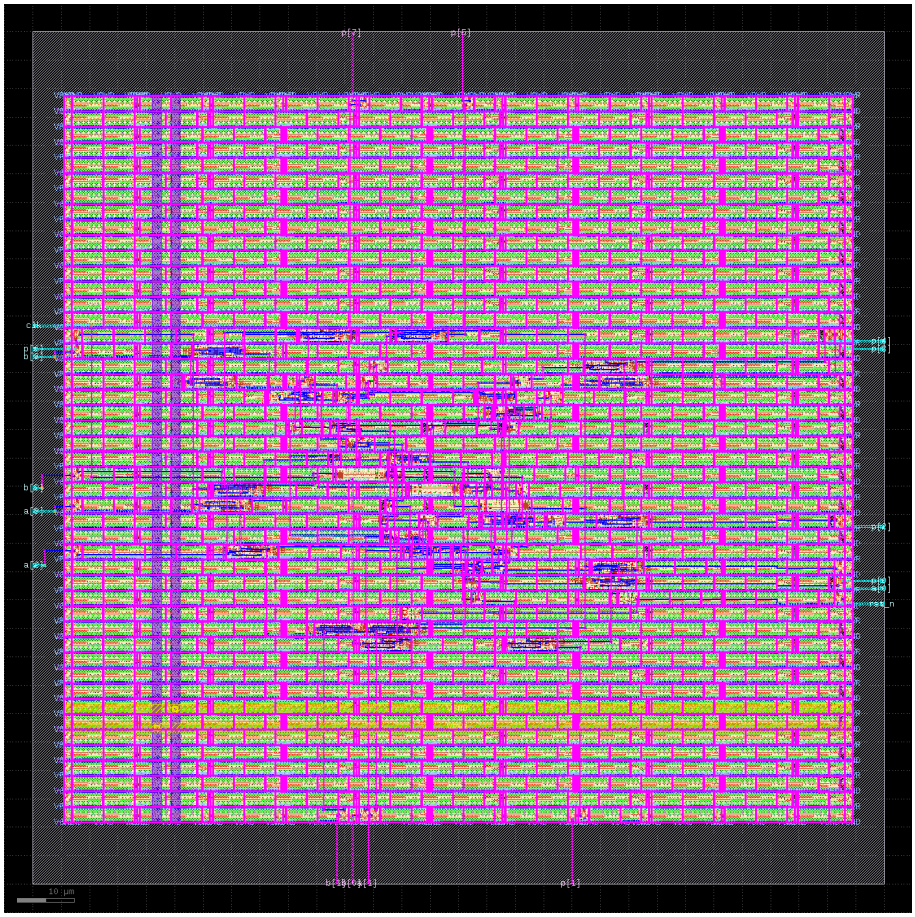


Figure 4: The whole 150 μm $\times$ 150 μm die. The cell rows fill the core, the power grid is seen as vertical and horizontal stripes, and the input and output pins (`clk`, `rst_n`, `a`, `b` and `p`) are on the die edges. The signal routing is in the middle. The scale bar at the bottom left is 10 μm.

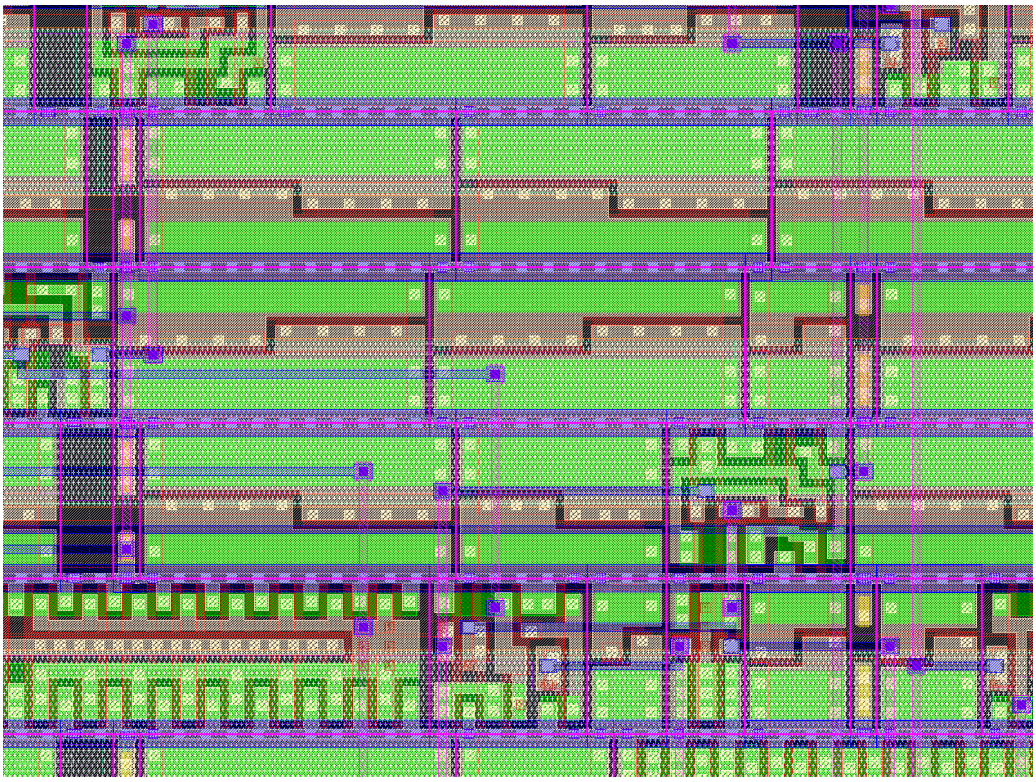


Figure 5: A zoomed area of about 18 μm × 12 μm in the middle of the die. Rows of standard cells run horizontally, with power rails between the rows. Horizontal and vertical metal wires cross over the cells, and the small squares are vias that connect the metal layers.

10.8 Post-route F_{max} compared with Part C

The post-route F_{max} is $1000/(7.0 - 0.335) = 1000/6.665 \approx$ **150.0** MHz. In Part C, F_{max} was **385.2** MHz, so the real layout is about 2.6 times slower than the ideal netlist. The main reasons are in Table 16.

Table 16: Why post-route timing is slower than the Part C analysis.

Effect	Part C	Part D signoff
Corner	typical (25°C, 1.8 V)	worst of several, including slow (100°C, 1.6 V)
Gate speed	`maj3_1` took 0.375 ns	a `maj3_1` took 0.89 ns in the slow corner
Wires	ideal, zero delay	extracted resistance and capacitance of 2,375 μm of metal
Clock	ideal, zero skew	real clock tree with skew
Clock uncertainty	none	0.25 ns taken from the period
Flip-flop setup	0.135 ns	0.293 ns (slow corner)

At 3.5 ns the typical-corner slack in the real layout was +0.568 ns. This gives a typical-corner post-route limit of about $1000/(3.5 - 0.568) \approx 340$ MHz, which is not far below the Part C value. So the biggest reason for the drop is not the wires or the clock tree, but the slow corner that signoff must meet. (This is a rough comparison, because every run synthesizes the design again.)

10.9 Observations

- **Routing needed one repair pass.** In the first iteration the router found 22 DRC violations, and the second iteration removed all of them. This is normal router behaviour, and it shows what DRC looks for.
- **Wire length.** The final routed length of 2,375 μm is about 5% more than the estimate before routing (2,262.63 μm), because detailed routing has to go around obstacles that the estimate ignores. The longest single net was 153.24 μm.
- **Small design, fixed die.** The cells use only a small part of the 150 μm $\times$ 150 μm die, so wires are short and congestion is not a problem. The price is the large number of filler cells.
- **Hold timing was not a problem.** The metrics show zero hold-repair buffers and zero setup-repair buffers. So the 19 buffers we counted were not added by the hold or setup repair steps. We think the optimizer added them for slew and load.
- **Clean signoff.** The final run has 0 DRC errors (Magic and KLayout agree), 0 LVS errors of every kind, and a positive worst setup slack.
- **Lesson.** A design can look fast in an ideal analysis and still need a much longer clock period after real corners and wires are included. The period from the simple rule ($1.2 \times T_{min}$) was not enough here, which is why signoff exists.

11 Answers to the questions

Question 1

The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic, and why is the number not simply $16 + 12$*?*

Synthesis produced 73 cells, and 16 of them are flip-flops, so the logic has $73 - 16 = \mathbf{57}$ cells (405.389 μm^2). This is about double the $16 + 12 = 28$ that we might expect. Yosys flattened the design, and ABC then mapped the whole multiplier as one block of logic. The library has full-adder cells, but none was used in our netlist, so every adder was built from smaller cells such as `xor3`, `xnor3`, `maj3` and AND-OR-INVERT gates. Many of the 16 AND gates were also merged into nearby gates (`a21oi`, `nand4`, `o2111a` and so on), so they are no longer separate cells. The adders with a constant carry-in of 0 also became simpler. So the count of 57 comes from the choices of the tool and not from our block diagram.

Question 2

What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about $2N$ *full-adder delays?*

The critical path starts at the input-register flip-flop for `b_r[0]` and ends at the output-register flip-flop for `p[6]`. There are **8 logic cells** between them: `and2_0`, `or3_1`, `and4_1`, `o2111a_1`, `o21ai_0`, `xnor3_1`, `maj3_1` and `xor3_1`. The data arrives at 2.460 ns (0.395 ns clock-to-Q plus 2.065 ns of logic), and the setup time is 0.135 ns, so $T_{min} = 2.596$ ns. The count of 8 is the same as the estimate $2N = 8$, but the cells are not eight full adders. Only `xnor3_1`, `maj3_1` and `xor3_1` look like adder cells, and they take 0.827 ns, about 40% of the logic delay.

The average cell delay is only about 0.26 ns, which is much less than a ripple full adder, because ABC merged adder and AND logic into complex gates.

Question 3

What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

The 16 flip-flops take $16 \times 25.024 = 400.384$ μm^2 out of 805.773 μm^2, which is **49.69%**, almost half. The percentage is large because the arithmetic itself is small. The 57 logic cells are only about 405 μm^2, and most of them are small gates. The registers are a fixed cost, because every input bit and every output bit needs its own flip-flop ($4 + 4 + 8$). One `dfrtp_1` (25.024 μm^2) is about 6.7 times bigger than a `nand2_1` (3.754 μm^2). So in a small design like this, storing the data costs as much area as computing with it. In a bigger arithmetic block the share of the registers would be smaller.

Question 4

Why is the post-route F_{max} lower than the F_{max} from Part C? Which kinds of cells did the flow add, and why is each kind needed?

The post-route F_{max} is $1000/(7.0-0.335) =$ **150.0** MHz, and the Part C value is **385.2** MHz. The two analyses are not the same. Part C used the typical corner with ideal wires and an ideal clock. The signoff also checks the slow corner (100°C, 1.6 V), where our `maj3` cell took 0.89 ns instead of 0.375 ns. It also includes real wires, a real clock tree with skew, a 0.25 ns clock uncertainty and a longer setup time (0.293 ns instead of 0.135 ns). The slow corner is the biggest part, because at 3.5 ns the same layout still had +0.568 ns of slack at the typical corner. The flow added these cells: 245 **tap cells** that connect the wells and substrate to the power rails and prevent latch-up; 1,313 **decap cells** and 249 **fill cells** that fill the empty row space and keep the power rails and layer density continuous (the decap cells also keep the supply steady); 3 **clock buffers** so that all 16 flip-flops get the clock at nearly the same time; and about 19 **timing-repair buffers** that fix slew and setup problems on long or heavily loaded nets.

Question 5

What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

A DRC (design rule check) compares the layout shapes with the design rules of the foundry, such as minimum width, minimum spacing and enclosure, so that the shapes can be printed and etched correctly. An LVS (layout versus schematic) check extracts the transistors and connections from the layout and compares them with the netlist, to confirm that the layout is the circuit we designed. The two checks find different mistakes. A layout can pass DRC and still be the wrong circuit, for example with a missing wire or a short between two nets. A layout can also be the right circuit and still break a rule, so it cannot be manufactured well. A chip is safe to send for manufacturing only when it can be built (DRC) and is correct (LVS). In our final run both were clean: 0 DRC errors from the router, Magic and KLayout, and 0 LVS errors of every kind.

Question 6

Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

An N-bit array multiplier has N^2 AND gates and $N(N-1)$ full adders, and its longest path is about $2N$ full-adder delays. For $N = 8$ this gives 64 AND gates (instead of 16) and $8 \times 7 = \mathbf{56}$ full adders (instead of $4 \times 3 = 12$), which is about 4.7 times as many. The flip-flops go from 16 to 32 (8 + 8 on the inputs and 16 on the product). The critical path goes from $2 \times 4 = 8$ to $2 \times 8 = \mathbf{16}$ full-adder delays, so the logic delay doubles. Using our Part C logic delay of 2.065 ns, the 8-bit logic needs about $2 \times 2.065 = 4.13$ ns. The register overhead stays the same (0.395 ns clock-to-Q and 0.135 ns setup, 0.53 ns together), so $T_{min} \approx 4.13 + 0.53 = \mathbf{4.66}$ ns, which is about **215** MHz instead of 385 MHz. So the area grows with N^2, but the delay grows only linearly. This is only an estimate, because the synthesis tool would map a bigger design differently.

12 Conclusion

What we built and found. We designed a 4-bit unsigned array multiplier from 16 AND gates and 12 full adders, with registers on the inputs and the output, and took it through the whole open-source flow on SKY130. The design is correct. It passed a test of all 256 input combinations at RTL level and again at gate level, and a deliberately broken version failed 95 of 256 tests, so the testbench can find mistakes. Synthesis gave 73 standard cells with a cell area of 805.773 μm^2, and about half of it is flip-flops. Static timing analysis at the typical corner gave a minimum period of 2.596 ns (385 MHz), with a critical path from `b_r[0]` to `p[6]` through 8 logic cells. Place and route gave a DRC-clean and LVS-clean layout at a clock period of 7.0 ns, with a worst setup slack of +0.335 ns, which is about 150 MHz.

Table 17: The same design at each stage.

Stage	Cells	Area	Speed	What the stage adds
RTL (Part A)	16 AND + 12 full adders + 16 flip-flops	n/a	n/a	Function only
Synthesis (Part B)	73	805.773 μm^2	n/a	Real cells and area
Typical-corner STA (Part C)	73	805.773 μm^2	385.2 MHz	Cell delays, ideal wires and clock
Place and route (Part D)	1,896 (334 without decap and fill)	core 17,759.5 μm^2	150.0 MHz	Wires, clock tree, slow corner, filler

What we learned.

1. *A gate count in RTL is not what ends up in silicon.* Synthesis changed our 28 gates and adders into 57 differently shaped cells, so reading the netlist taught us more than reading the RTL.

2. *Timing depends on how closely we look.* The same design runs at 385 MHz in an ideal analysis and at 150 MHz after signoff. The slow corner is the largest part of the gap, and

the real wires and clock tree add more. The first period from the assignment rule (3.5 ns) failed by 2.1 ns, and we needed eight runs to find a period that passed.

3. *Overheads are large in small designs.* Half of the synthesized area is flip-flops, and in the layout more than 80% of the cells are filler (decap and fill) around a very small amount of logic.

4. *Checking should be automatic.* A self-checking testbench that we could use again at the gate level, and the DRC and LVS checks in the flow, meant we did not have to depend on looking at waveforms.

Limitations. Part C used only one corner and an ideal clock, and we did not run a hold analysis ourselves. We searched for the clock period in steps of 0.5 ns, so 7.0 ns is a passing period but not always the smallest one. The runs from 5.5 to 6.5 ns also show that the result does not always improve with a longer period. The cell count in Part B and Part D cannot be compared exactly, because LibreLane synthesizes the design again with its own settings. We did not find the real reason why the extra period stopped helping after 5.5 ns, and our explanation in Section 10 is only a guess.

Possible improvements. To make the design faster, we could use a faster adder structure, such as a Wallace or Dadda tree with a fast final adder, or add pipeline registers inside the array (this costs more latency and more flip-flops). To make it smaller, we could let the tool choose the die size from the cell area instead of using a fixed 150 μm $\times$ 150 μm die, which would remove most of the decap and fill cells.

Reproducibility. Every step is a command in the README, and every file used in this report is in the repository with the structure that the assignment asks for.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson. Chapter 11, datapath subsystems (adders and multipliers).

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson. Arithmetic building blocks and timing.

[3] Icarus Verilog: steveicarus.github.io/iverilog

[4] GTKWave: gtkwave.github.io/gtkwave

[5] Yosys: yosyshq.readthedocs.io/projects/yosys

[6] OpenSTA: github.com/parallaxsw/OpenSTA (manual: doc/OpenSTA.pdf)

[7] LibreLane: github.com/librelane/librelane

[8] OpenROAD: openroad.readthedocs.io

[9] KLayout: klayout.de/doc.html

[10] Magic and Netgen: opencircuitdesign.com/magic and opencircuitdesign.com/netgen

[11] SkyWater SKY130 PDK: skywater-pdk.readthedocs.io

[12] IIC-OSIC-TOOLS: github.com/iic-jku/IIC-OSIC-TOOLS